

Fundamentos de Programación

Tema 4: Estructuras de datos

Contenidos

1.- Introducción	2
2.- Estructuras de datos	3
3.- El Java Collection Framework	4
4.- La interfaz Collection<T>	7
5.- La interfaz List<T>	7
6.- Recorrido de una lista	10
7.- El for mejorado	13
8.- Listas mutables e inmutables	14
9.- Las clases envoltorio	15
10.- Arrays	16
11.- Tablas	22
12.- La interfaz Set<T>.....	27
13.- La interfaz Map<K,V>	28

Tema

4

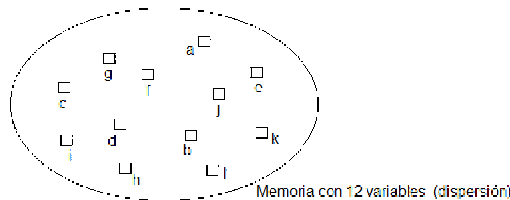
ESTRUCTURAS DE DATOS

La programación estructurada y la programación orientada a objetos son las herramientas que nos permiten escribir el código fuente de cualquier programa. Sin embargo, también es necesario saber organizar los datos en la memoria correctamente.

1.- Introducción

Como sabemos, la única forma de que un programa pueda trabajar con datos es tenerlos guardados en variables. Así es como hemos trabajado hasta este momento: cada vez que necesitamos un dato, creamos una variable del tipo adecuado donde lo almacenamos. La variable siempre ocupa un espacio dentro de la memoria del ordenador.

En la práctica los programas suelen trabajar con grandes cantidades de **datos similares**, y entonces surge este problema: llega un momento en el que tenemos muchas variables, cada una con su propio nombre, y se hace difícil trabajar con todas a la vez cuando necesitamos hacer tareas en las que intervienen todas las variables.



Por ejemplo, supongamos que tenemos que hacer un programa que calcule la nota media de un alumno que ha hecho 3 exámenes. Creamos las variables y calculamos su media:

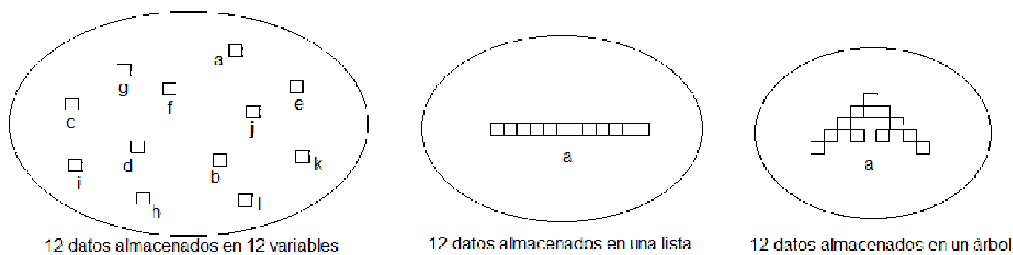
```
1. public class Media{
2.     public static void main(String[] args){
3.         Scanner sc=new Scanner(System.in);
4.         int nota1=sc.nextInt();
5.         int nota2=sc.nextInt();
6.         int nota3=sc.nextInt();
7.         double media=(nota1+nota2+nota3)/3.0;
8.         ct.println(media);
9.     }
10. }
```

Este programa es correcto, pero tiene dos importantes problemas de mantenimiento:

- El número de notas está limitado a 3. Cada vez que éste aumente o disminuya, hay que modificar y volver a compilar el programa, lo cual, no es ni cómodo ni práctico.
- Si el número de notas es muy grande se necesitan muchas variables, y entonces el cálculo de la media se hace muy pesado, con mucha más probabilidad de equivocarnos

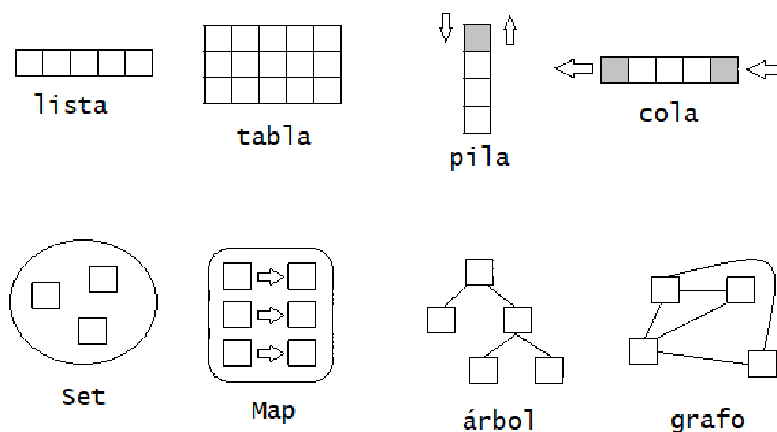
En resumen, hay situaciones donde hay que trabajar con muchos datos "similares", y no es adecuado utilizar variables para guardarlos, porque tendríamos muchas variables y entonces se haría muy difícil o imposible su tratamiento conjunto.

En estas situaciones lo que se hace es utilizar una **estructura de datos**, que va a ser **un solo objeto** que agrupa en la memoria todos los datos del problema. De esta forma, en lugar de tener cientos de variables, solamente tendremos una sola, que será la colección. La colección tendrá métodos para añadirle datos, eliminarlos, consultar cuantos hay, etc.



2.- Estructuras de datos

De forma simplificada podemos decir que una **estructura de datos** es una forma concreta de almacenar en la memoria un conjunto de datos. Hay muchísimas estructuras, que se diferencian en la forma en que se almacenan y procesan los datos que contienen. Las estructuras más importantes son:



- **Lista:** Es una secuencia ordenada de datos, en los que cada uno ocupa una posición.
- **Tabla:** Es una disposición de datos en filas y columnas.
- **Pila:** Estructura que se forma cuando los datos se añaden uno encima de otro, de forma que siempre tenemos acceso al que está encima de todos.
- **Cola:** Estructura en la que los datos se añaden uno a continuación de otro, de forma que el primero en salir de la cola es el primero que llegó.
- **Set:** Colección de objetos diferentes en la que no se tiene en cuenta el orden.
- **Map:** Es una estructura que almacena parejas de datos asociados.

- **Árbol:** Estructura en la que hay un dato padre (**raíz**) del que cuelgan datos hijos. A su vez, cada hijo puede tener varios hijos y así sucesivamente.
- **Grafo:** Estructura en la que cualquier dato puede estar relacionado con los demás sin ningún tipo de restricción.

Cada una de estas estructuras está programada en una **clase** que suele venir en las librerías de cada lenguaje de programación. Dependiendo del problema que tengamos que resolver, será necesario elegir la estructura de datos que mejor nos convenga. Muchos problemas admiten soluciones diferentes si se usan estructuras distintas.

Ejercicio 1 : ¿Qué estructura de datos usarías para almacenar la siguiente información?

- a) El horario de clases de 1º DAW
- b) El conjunto de canciones que un equipo de música debe reproducir secuencialmente.
- c) Las notas de los alumnos de una clase, ordenadas de mayor a menor.
- d) Las personas que están esperando en la taquilla de un cine.
- e) Las cartas de una baraja antes de empezar a sacarlas de una en una.
- f) Un conjunto de tareas programadas, donde cada una comienza a una hora prevista.
- g) Los jugadores de un equipo de fútbol.
- h) Los sprites que aparecen en la pantalla de un videojuego
- i) Los platos sucios que hay que lavar en un restaurante.
- j) Una tarrina de CDs.
- k) Un conjunto de personas en el que hay unos que conocen a otros.
- l) El sistema de archivos de un medio de almacenamiento (disco duro, cd, etc).
- m) El bombo que guarda las bolas para jugar a la lotería.
- n) El conjunto de archivos que debe descargar un navegador.
- o) Los billetes que expende un cajero automático.
- p) El conjunto de routers que forman Internet

3.- El Java Collection Framework

Una de las librerías más importantes de Java es el **Java Collection Framework**, ya que proporciona clases para usar listas, pilas, colas, Set y Map, y además contiene los medios para que un programador de clases pueda programar su propia estructura de datos de forma que se ajuste a los estándares de Java.

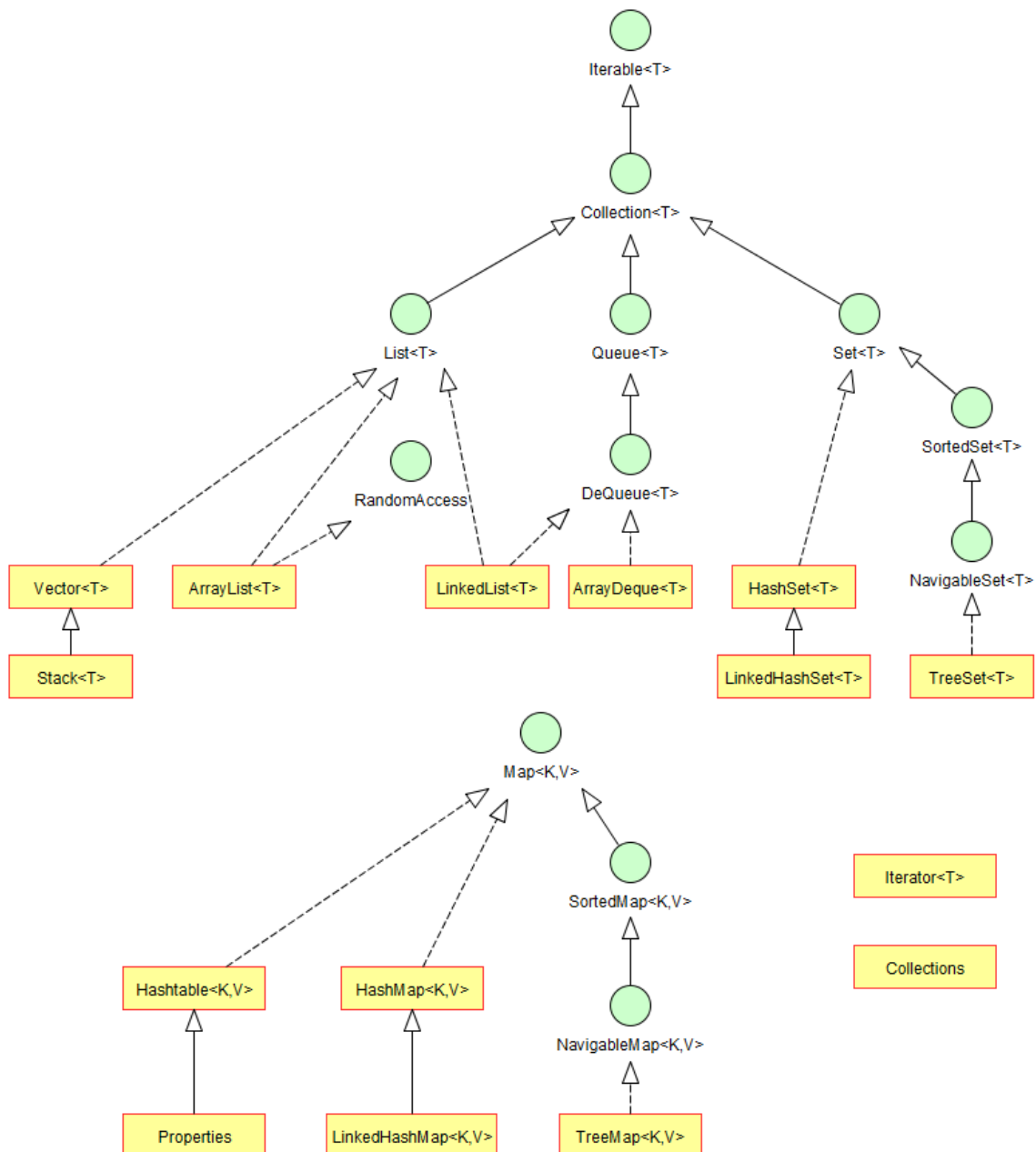
Antes de comenzar a estudiar esta librería, vamos a indicar dos cuestiones muy importantes sobre ella:

- Las clases de esta librería usan generics: esto quiere decir que el nombre de cada clase termina con los ángulos < > que encierran el tipo de dato de los objetos que contiene. Por ejemplo, si escribimos **ArrayList<String>** quiere decir que tenemos una lista que guarda String. En cambio **ArrayList<LocalDate>** sería una lista que guarda dentro

objetos `LocalDate`¹. En la documentación de las clases, suele escribirse **`ArrayList<T>`** para indicar una clase que contiene datos de un tipo cualquiera T.

- Las clases de esta librería solo guardan objetos. Por tanto, no es posible guardar tipos básicos en ellas. Por ejemplo, escribir `ArrayList<int>` nos daría un error de compilación. En el apartado 8 veremos cómo se soluciona este problema.

La librería del Java Collection Framework está formada por un árbol donde aparecen tipos de datos que son padre de los demás. De forma simplificada, esta librería se estructura así:



¹ Los ángulos pueden omitirse y dar lugar a las clases **raw type**. Por ejemplo, `ArrayList` (sin ángulos) sería una clase raw-type, que admite cualquier objeto. Estas clases son antiguas y su uso está desaconsejado por ser muy propensas a que el programador cometa errores utilizándolas. El uso de generics apareció para evitar usar las clases raw-type.

Como vemos, los tipos de datos que aparecen en el diagrama son de dos categorías:

- **Clases:** Son los rectángulos de color amarillo. Por ejemplo, **ArrayList<T>** es una clase.
- **Interfaces:** Son los círculos de color verde del diagrama. Por ejemplo, **List<T>** es una interfaz. Las interfaces² son un tipo especial de tipos de datos referencia que no son clases y surgen para poder hacer herencia múltiple. Por ahora nos basta con saber que las interfaces:
 - Son tipos de dato referencia, y por tanto, tienen métodos igual que las clases. Por ejemplo, List<T> es un tipo de dato referencia y tiene sus propios métodos. Lo que ocurre es que es una interfaz, y no es una clase.
 - No podemos crear objetos de ellas directamente. Una característica de las interfaces es que nunca tienen método constructor. Por tanto, la única forma de obtenerlas es mediante la **abstracción** o buscando un método que nos la de
 - Permiten la herencia múltiple. Por ejemplo, vemos que ArrayList<T> es una especie de “hijo” de la interfaz RandomAccess y también de la interfaz List<T>. Aunque no sale en este diagrama, también puede ocurrir que una interfaz tenga varias interfaces padre.
 - Las clases se conectan con sus interfaces “padre” con flechas discontinuas, y cuando esto ocurre diremos que **“la clase implementa la interfaz”**. Por ejemplo, en el diagrama podemos ver que *“Vector<T> implementa la interfaz List<T>”*. En cambio, cuando una interfaz hereda de otra, si diremos que la interfaz es hija de la otra. Por ejemplo *“La interfaz List<T> es hija de la interfaz Collection<T>”*

En resumen:

- **Una interfaz puede tener varias interfaces padre.**
- **Una clase solo puede tener una clase padre.**
- **Una clase puede implementar varias interfaces.**

En todo caso, la relación que existe entre las clases y las interfaces sigue funcionando de la misma forma que en la herencia: el hijo recibe los métodos del padre.

Cuando trabajamos con el diagrama del Java Collection Framework, siempre crearemos objetos de las clases, porque son los que tienen método constructor.

Por ejemplo, si queremos crear una lista que contenga muñecos de Mario, deberemos crear un objeto de la clase **ArrayList<Mario>**, ya que no es posible crear directamente un objeto de tipo **List<Mario>**.

² No confundir estas interfaces con la “interfaz de usuario” que forman las ventanas de la aplicación.

4.- La interfaz Collection<T>

Collection<T> representa una colección de datos genérica y es la interfaz padre de las listas, pilas, colas y Set. Por tanto, sus métodos los encontraremos en todos los elementos del diagrama. De todos los métodos de Collection<T>, destacamos:

- **add:** Permite añadir un objeto a la colección.
- **remove:** Borra un objeto de la colección.
- **size:** Método que devuelve el total de objetos que hay en la colección.
- **clear:** Borra todos los objetos que hay en la colección.
- **contains:** Permite comprobar si un objeto se encuentra en la colección.
- **isEmpty:** Permite comprobar si la colección está vacía
- **toString:** Devuelve un String con todos los objetos que hay en la colección separados por comas.

5.- La interfaz List<T>

El tipo List<T> representa una lista de objetos. Por tanto, se trata de una secuencia ordenada de objetos donde cada uno se encuentra en una posición. Las listas son estructuras muy versátiles y prácticamente se pueden incluir en cualquier programa. Algunos ejemplos:

- En un programa de contabilidad, para registrar los ingresos de cada mes podemos formar una lista. Se supone que el primer dato serían los ingresos de enero, el segundo serían los de febrero, etc.

15000	12300	4900	16320
-------	-------	------	-------

- En un programa de agenda escolar podemos hacer una lista donde en cada posición se guardaría la información de los exámenes realizados y los que están pendientes.

examen de java	examen de fol	examen de java	examen de fol
10/1/2019	14/2/2019	27/2/2019	1/3/2019
nota: 6	nota: 8	sin hacer	sin hacer

- En un juego de fútbol, elegir la alineación de los jugadores daría lugar a una lista. En cada posición se guardaría un objeto con el jugador y la posición estaría relacionada con el número de su dorsal (los primeros dorsales ocuparían las primeras posiciones).



Cuando se trabaja con listas es vital saber distinguir lo que son las **posiciones** de los **datos** de la lista. Los datos son los objetos que guardamos en la lista, mientras que las posiciones son números enteros que nos dicen donde se encuentra cada objeto de la lista:



Como es usual en casi todos los lenguajes de programación, en Java las posiciones de las listas comienzan en el número **0**.

El tipo `List<T>` recibe de `Collection<T>` todos los métodos que vimos en el apartado anterior, pero además tiene otros, como por ejemplo:

- **get:** Permite obtener el elemento de la lista que ocupa una posición.
- **set:** Sustituye un elemento de la lista por otro
- **indexOf:** Devuelve la posición de la primera aparición en la que se encuentra un objeto. Para detectar si un objeto está en la lista, internamente usa **equals**.
- **lastIndexOf:** Devuelve la posición de la última aparición de un objeto.
- **remove:** Permite borrar un dato de la lista a partir del número de posición del dato.

En la práctica, cuando vamos a hacer un programa no podemos crear directamente un objeto de `List<T>`, sino que tenemos que acudir a alguna clase hija, siendo la más habitual **`ArrayList<T>`**. Esta clase tiene dos métodos constructores:

- **`ArrayList<T>()`:** Crea una lista vacía.
- **`ArrayList<T>(Collection<T> c)`:** Crea una lista con todos los objetos que tiene el `Collection<T>` que le pasamos como parámetro. Esto es muy útil cuando queremos obtener una lista a partir de los objetos que tiene cualquier otra colección, como por ejemplo una cola o un Set.

Por ejemplo, vamos a crear una lista vacía para guardar `String` y le vamos a añadir varios datos:

```
1. ArrayList<String> lista=new ArrayList<String>();
2. lista.add("Hola");
3. lista.add("Adios");
4. lista.add("Fin");
5. System.out.println("La lista tiene "+lista.size()+" datos");
```

El código anterior crea una lista que puede guardar `String`, le añade las palabras "Hola", "Adios", "Fin" y por último, muestra en pantalla su número total de objetos.

Aunque el código del ejemplo es correcto, es habitual mejorarlo de estas dos formas:

- 1) Podemos quitar el tipo de dato del generic que hay después de la palabra new y dejar solamente los ángulos <> (esto se llama “operador diamond”):

```
1. ArrayList<String> lista=new ArrayList<>();
2. lista.add("Hola");
3. lista.add("Adios");
4. lista.add("Fin");
5. System.out.println("La lista tiene "+lista.size()+" datos");
```

- 2) Como un ArrayList<String> es un List<String>, podemos usar **abstracción** para guardar el objeto creado en una variable de tipo List<String>, así:

```
1. List<String> lista=new ArrayList<>();
2. lista.add("Hola");
3. lista.add("Adios");
4. lista.add("Fin");
5. System.out.println("La lista tiene "+lista.size()+" datos");
```

☞ *¿Por qué se considera el uso de la abstracción como una mejora?* ArrayList<T> no es la única hija de List<T>. Hay muchísimas más, como por ejemplo LinkedList<T> o CopyOnWriteArrayList<T>. Al guardar la lista en el tipo genérico List<T>, nuestro programa se hace independiente del tipo de lista usado, y si en el futuro queremos cambiar el tipo de la lista, solo tendremos que tocar en una línea, facilitando así el mantenimiento. Por ejemplo, si queremos cambiar a una LinkedList<T>, solo tendremos que cambiar lo que hay después de la palabra new, dejando intacto el resto del programa:

```
1. public class Ejemplo{
2.     public static void main(String[] args){
3.         List<String> lista=new LinkedList<String>();
4.         lista.add("Hola");
5.         lista.add("Adios");
6.         lista.add("Fin");
7.         System.out.println("La lista tiene "+lista.size()+" datos");
8.     }
9. }
```

Usar List<T> en lugar de ArrayList<T> hace que puedas olvidarte del tipo de lista que estás usando en el programa, y puedas cambiarla por cualquier otro tipo cuando sea necesario.

Por último, si queremos mostrar en pantalla todos los objetos de la lista es suficiente llamar al método **toString**, que es llamado automáticamente cuando ponemos un objeto dentro de System.out.println.

```
1. List<String> lista=new ArrayList<>();
2. lista.add("Hola");
3. lista.add("Adios");
4. lista.add("Fin");
5. System.out.println("La lista tiene "+lista.size()+" datos");
6. System.out.println(lista); // también sirve System.out.println(lista.toString());
```

La salida del programa muestra por pantalla **[Hola,Adios,Fin]**, que es la salida por defecto que tiene el método toString.

Si quisiéramos mostrar una vista diferente (por ejemplo, que se vea el número de posición de cada objeto, o mostrar cada objeto en un renglón distinto) deberíamos programarla usando los métodos de List<T>, como practicaremos más adelante en los ejercicios.

Ejercicio 2 : Haz un programa que cree una lista de Strings llamada “mensajes” y muestre un menú con las siguientes opciones:

- 1) Añadir mensaje: Cuando es seleccionado, pedirá al usuario que escriba un mensaje para guardarlo en la lista y entonces se añadirá a la lista de mensajes.
- 2) Consultar tamaño: Muestra el número de mensajes que hay en la lista.
- 3) Consultar mensaje: Cuando se selecciona, pueden pasar dos cosas:
 - a. Si la lista está vacía, se mostrará “No hay mensajes” y se volverá al menú.
 - b. Si la lista no está vacía, el programa preguntará el número del mensaje que se desea consultar y el usuario deberá introducir un número entre 0 y la última posición válida de la lista. Si se introduce un número fuera de ese rango, el programa avisará de ello. En caso contrario, se mostrará el mensaje indicado.
- 4) Comprobar mensaje: Se pedirá al usuario que escriba un mensaje y el programa mostrará si el mensaje está o no en la lista. En caso de que si esté, se mostrará también su número de posición.
- 5) Consultar todos los mensajes: Se mostrarán todos los mensajes de la lista.
- 6) Borrar todo: Cuando se selecciona se borran todos los mensajes de la lista.

Ejercicio 3 : Haz un programa que reciba un número como argumento de la línea de comandos. A continuación hará lo siguiente:

- Si el programa no recibe ningún número, o se recibe un número no válido se mostrará el mensaje “Debe introducir un número entre 1 y 12” y finalizará.
- El programa creará una lista y la rellenará con objetos LocalDate para todos los días del mes cuyo número se ha pasado como parámetro.
- Se mostrará por pantalla la lista y también su tamaño.

6.- Recorrido de una lista

Cuando tenemos una lista, es muy frecuente tener que realizar un **recorrido** de ella. Hacer un recorrido significa dar una pasada a la lista visitando todos sus objetos y haciendo algo con cada uno. Por ejemplo, supongamos que tenemos una lista con los objetos Empleado de una empresa y queremos subirles el sueldo a todos. Debemos dar una pasada por todos los empleados de la lista y subir el sueldo a cada uno.

Las listas admiten dos tipos de recorrido:

- Recorrido hacia adelante: Se parte de la posición 0 y se avanza hasta la última.
- Recorrido hacia atrás: Se parte de la última posición y se retrocede hasta la posición 0.

En ambos casos, los recorridos se hacen con un bucle **for** cuya variable contadora “i” genere las posiciones que queremos visitar. Dentro del for, se usa el método **get** para recuperar el dato de la lista que ocupa la posición visitada.

Por ejemplo, si queremos hacer un recorrido hacia adelante, usaríamos un for para generar los números entre 0 y el tamaño de la lista (que nos lo devuelve su método **size**) así:

```
1. List<Empleado> lista= // obtenemos la lista de objetos Empleado
2. // recorremos hacia adelante la lista de empleados
3. for(int i=0;i<lista.size();i++){
4.
5. }
```

Para terminar, dentro del for usamos **get** para extraer de la lista el empleado que está en la posición “i” y así poder subirle el sueldo:

```
1. List<Empleado> lista= // obtenemos la lista de objetos Empleado
2. // recorremos hacia adelante la lista de empleados
3. for(int i=0;i<lista.size();i++){
4.     /* obtengo el empleado que está en la posición "i" de la lista
5.     (i cambia en cada vuelta y por tanto, en cada vuelta saldrá un empleado distinto) */
6.     Empleado e = lista.get(i);
7.     e.subirSueldo(100);
8. }
```

El bucle for que hemos realizado realmente funciona y le sube el sueldo a todos los empleados, ya lo que hace es como si escribiésemos esto (suponemos tres empleados):

```
1. // i=0
2. Empleado e = lista.get(0);
3. e.subirSueldo(100);
4. // i=1
5. Empleado e = lista.get(1);
6. e.subirSueldo(100);
7. // i=2
8. Empleado e = lista.get(2);
9. e.subirSueldo(100);
```

También es posible hacer el recorrido hacia atrás, modificando el for para que se generen las posiciones al revés, comenzando en la última (lo que devuelve el método **size** menos uno) hasta llegar a 0. Sería así:

```
1. List<Empleado> lista= // obtenemos la lista de objetos Empleado
2. // recorremos hacia atrás la lista de empleados
3. for(int i=lista.size()-1;i>=0;i--){
4.     /* obtengo el empleado que está en la posición "i" de la lista
5.     Empleado e = lista.get(i);
6.     e.subirSueldo(100);
7. }
```

En este caso el bucle for sería equivalente a realizar estas acciones:

```
1. // i=2
2. Empleado e = lista.get(2);
3. e.subirSueldo(100);
4. // i=1
5. Empleado e = lista.get(1);
6. e.subirSueldo(100);
7. // i=0
8. Empleado e = lista.get(0);
9. e.subirSueldo(100);
```

En una lista se pueden hacer más recorridos diferentes. Por ejemplo, recorrer la lista alternando los extremos hasta llegar al centro, etc. Cualquier tipo de recorrido se realizaría mediante bucles for adecuados. En general, usando el for es posible programar cualquier recorrido de una lista que nos podamos imaginar.

Ejercicio 4 : Realiza un programa que cree una lista con 10 palabras. A continuación el programa nos mostrará cada palabra de la lista en un renglón diferente.

Sugerencia: Usa el método **split** de la clase **String**

Ejercicio 5 : Realiza un programa que cree una lista y le añada 10 muñecos de Mario repartidos sobre la línea central de la pantalla. A continuación, todos los muñecos que ocupen una posición par de la lista comenzarán a andar hacia el punto de coordenadas (0,0) y los que ocupen una posición impar se dirigirán hacia el punto (1024,0).

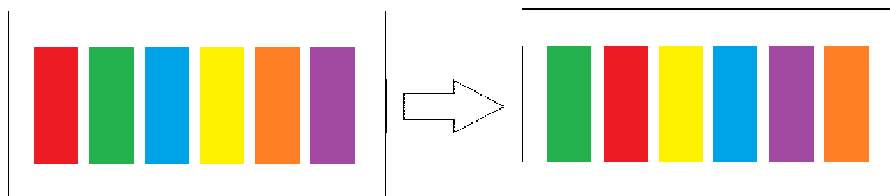
Ejercicio 6 : Realiza un programa que cree una lista vacía y a continuación, pregunte 10 números al usuario. Cada número introducido se guardará en la lista. Al terminar de introducir el último número, el programa recorrerá la lista y obtendrá el mayor y menor número de ella.

Ejercicio 7 : Usa la librería ReproductorMusica para realizar un programa que cree una lista con 10 canciones mp3. A continuación se preguntará al usuario “Seleccione una canción:” y se mostrarán todos los títulos de todas las canciones de la lista (cada uno en un renglón diferente y además acompañado del número de su posición). El usuario podrá elegir el número de canción que desea reproducir y en ese momento, la canción comenzará a sonar de forma asíncrona y se volverá al menú. El programa también tendrá una opción para salir.

Ejercicio 8 : Haz un programa que cree una lista rellena con 50 depósitos de agua vacíos, con un máximo de 100 litros de capacidad. El programa mostrará este menú:

1. Dibujar depósito → *Pregunta el número de un depósito de agua y lo dibuja*
2. Rellenar depósito → *Pregunta el número de un depósito de agua y la cantidad de agua que llenarlo. El programa rellenará dicho depósito la cantidad de litros indicada.*
3. Vaciar depósito → *El programa pregunta el número de un depósito y lo vacía.*
4. Añadir depósito → *Se añade a la lista un nuevo depósito vacío con 100 litros de capacidad, y se mostrará la posición que tiene dicho depósito en la lista de depósitos*
5. Salir → *El menú se repite hasta elegir esta opción*

Ejercicio 9 : Usa la Consola DAW para hacer un programa que muestre en pantalla 6 rectángulos de colores. Al pulsar una tecla, los rectángulos se intercambiarán de dos en dos.



Sugerencia: Puedes usar dos listas, una para almacenar objetos Rectangle con los rectángulos y otra lista para almacenar sus colores.

Mejora: ¿Serías capaz de cambiar el 6 por un número par N introducido por el usuario?

7.- El for mejorado

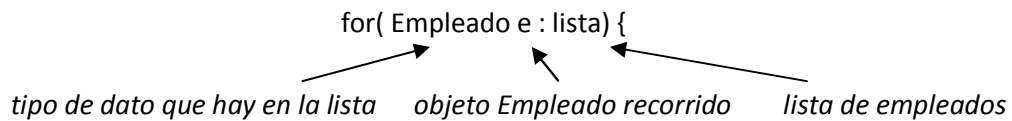
El **for mejorado**³ es una variación del bucle for que nos permite recorrer hacia adelante una lista, sin tener que generar las posiciones de los datos. El for mejorado se encarga automáticamente de ir sacando los objetos de la lista.

En el apartado anterior vimos que con un bucle for normal y corriente podemos recorrer hacia adelante una lista. El contador del for “i” recorre las posiciones de los datos:

```
1. List<Empleado> lista= // obtenemos la lista de objetos Empleado
2. // recorremos hacia adelante la lista de empleados
3. for(int i=0;i<lista.size();i++){
4.     /* obtengo el empleado que está en la posición "i" de la lista
5.     (i cambia en cada vuelta y por tanto, en cada vuelta saldrá un empleado distinto) */
6.     Empleado e = lista.get(i);
7.     e.subirSueldo(100);
8. }
```

El for que hemos hecho podría leerse así: “Para cada posición i de la variable lista, saco el empleado e que está en la posición i y le subo el sueldo, avanzando i hacia adelante”

El for mejorado consiste en pensar de forma más sencilla, así: “*Para cada empleado e de la variable lista, le subo el sueldo*”. Esto lo vamos a escribir así.



En el for mejorado el “contador” es un objeto de la lista, y no un número

Usando el for mejorado, el código fuente del ejemplo anterior se escribiría así:

```
1. List<Empleado> lista= // obtenemos la lista de objetos Empleado
2. // recorremos hacia adelante la lista de empleados
3. for(Empleado e : lista){
4.     // en cada vuelta, "e" es un empleado distinto
5.     e.subirSueldo(100);
6. }
```

En general, usar un for mejorado es más sencillo que hacer un bucle for que genere posiciones. Pero el for mejorado tiene algunos inconvenientes:

- Solo sirve para realizar recorridos hacia adelante.
- No es posible realizar recorridos con interrupción. Por ejemplo, a veces puede ser que nos interese visitar los datos de la lista hasta encontrar alguno que nos interese y luego ya no seguir visitando más. Para hacer eso necesitaríamos interrumpir las repeticiones, y eso no se puede hacer con el for mejorado.
- No es posible consultar la posición que estamos recorriendo, por lo que no podremos hacer bucles en los que necesitemos ese dato.
- No es posible hacer asignación en la variable que estamos recorriendo para cambiarla.

³ En inglés, **enhanced for**

El for mejorado es muy importante, ya que con él podemos recorrer cualquier objeto que sea **Iterable<T>**. Si miramos el diagrama de la página 5 veremos que **Iterable<T>** es el tipo situado justo en lo alto, por lo que todas las clases de la Java Collection Framework (excepto las derivadas de Map) pueden ser recorridas con un for mejorado.

Ejercicio 10 : Usa la Consola DAW para realizar un programa que cree una lista con 20 objetos Rectangle de tamaño 50x50 y posición aleatoria. A continuación, se mostrará este menú:

1. Dibujar los rectángulos sin ningún color: El ordenador dibujará todos los rectángulos de la lista sin rellenar de color, utilizando el for mejorado para recorrer la lista.
2. Dibujar los rectángulos con color rojo: El ordenador dibujará todos los rectángulos rellenos con color rojo, utilizando el for mejorado para recorrer la lista.
3. Dibujar los rectángulos con color verde: El ordenador dibujará todos los rectángulos rellenos con color verde, utilizando el for mejorado para recorrer la lista.
4. Salir: El programa termina.

Ejercicio 11 : Haz un programa en el que haya una lista rellena con las 5 palabras que tú quieras. El ordenador recorrerá la lista y al final mostrará cuál es la palabra de mayor longitud.

8.- Listas mutables e inmutables

Actualmente, cuando se trabaja con programación orientada a objetos, se está haciendo mucho énfasis en el concepto de **mutabilidad**

Un objeto es **mutable** cuando es posible modificarlo, e **immutable**, cuando no

Algunas clases mutables que hemos visto son el DepositoAgua (porque es posible añadir y retirar agua al depósito, modificando su estado interno) o la Caja (porque podemos cambiar el mensaje de la caja). En cambio, el String es immutable, ya que no hay ningún método que nos permita modificarlo (por ejemplo, toUpperCase no modifica el String, sino que devuelve un String nuevo con su conversión a mayúsculas). Lo mismo le sucede a la clase LocalDate. Hoy día se prefieren, siempre que sea posible, clases inmutables, ya que se ha visto que su uso es menos propenso a errores.

La clase **ArrayList<T>** es mutable, ya que es posible añadir y retirar objetos a ella. En Java no hay ninguna clase que represente una lista inmutable, pero podemos usar el método estático **of** de la interfaz **List<T>** para obtener un **List<T>** que sea inmutable.

Por ejemplo, si queremos hacer una lista inmutable con los días de la semana escribiremos:

```
1. List<String> dias = List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");
```

La lista obtenida de esta forma es inmutable, y eso significa que los métodos que la modifican (**add**, **remove**, etc) lanzarán una excepción.

9.- Las clases envoltorio

Como ya hemos visto, una restricción que tienen las clases del Java Collection Framework es que, debido a como están programadas, no es posible guardar en ellas tipos básicos. ¿Qué se puede hacer entonces si necesitamos, por ejemplo, una lista de números?

La solución está en las **clases envoltorio**, que son clases que sustituyen a los tipos básicos. Es decir, por cada tipo básico tenemos una clase que lo representa:

Tipo básico	Clase envoltorio
boolean	Boolean
short	Short
char	Character
int	Integer
long	Long
float	Float
double	Double

Java tiene una característica llamada **autoboxing**, que significa que los tipos básicos y sus envoltorios son intercambiables, es decir, podemos poner un tipo básico en cualquier lugar donde se necesite un objeto envoltorio y al revés, se puede poner un objeto envoltorio en cualquier lugar donde se necesite su tipo básico. ¿Por qué es tan importante esto? Veámoslo con ejemplos:

- 1) Podemos crear un objeto envoltorio asignando directamente su valor de tipo básico:

```
1. Integer a = 8;  
2. Boolean b = false;  
3. Double c = 9.35;  
4. Character d = 'x';
```

- 2) Podemos hacer operaciones matemáticas con objetos envoltorio y guardar el resultado en tipos básicos o incluso en otros objetos envoltorio.

```
1. int a = 5;  
2. Integer b = 9;  
3. Integer c = ( a + b ) * a / 2;  
4. int d = c + 40 - Math.sqrt(b);
```

- 3) Podemos crear una lista que guarde objetos Integer (tipo envoltorio) y rellenarla con datos de tipo int (tipo básico).

```
1. List<Integer> lista = new ArrayList<>();  
2. lista.add(5);  
3. lista.add(93);  
4. lista.add(44);
```

- 4) Si tenemos una lista que guarda objetos Integer, su método get nos dará objetos Integer, pero podemos guardarlos directamente en variables de tipo int (tipo básico)

```
1. List<Integer> lista = new ArrayList<>();  
2. lista.add(5);  
3. int dato = lista.get(0); // recupera un 5 y lo guarda en la variable "dato"
```

Los tipos envoltorio son objetos, y por tanto tienen métodos que podemos utilizar. No obstante, ocupan más espacio en la memoria que los tipos básicos y hacer operaciones con ellos es más lento, por lo que solo deben usarse cuando sean necesarios.

Ejercicio 12 : Realiza un programa en el que se cree una lista rellena con 10 números aleatorios entre 0 y 100. A continuación, el programa preguntará al usuario un número por teclado y nos dirá si el número que ha escrito está o no en la lista.

Ejercicio 13 : Crea una lista con 50 números enteros rellenos aleatoriamente. El programa usará la clase **Collections** para ordenarla de menor a mayor y luego la mostrará por pantalla.

10.- Arrays

Los **arrays** son estructuras de datos que se encuentran en muchos lenguajes de programación. Los primeros lenguajes, como C, Pascal o Basic ya tenían los arrays.

Un array es una lista mutable de tamaño fijo, es decir, una vez creado, su cantidad de elementos es constante y no se puede cambiar, por lo que no es posible que aumente o disminuya de tamaño, como le pasa al `ArrayList<T>`. Aunque esto parece una desventaja, el uso de arrays tiene ventajas como estas:

- Trabajar con un array es más eficiente a nivel interno que usar un `ArrayList<T>`, ya que consume menos memoria y además el acceso a sus elementos es más rápido.
- Podemos crear arrays tanto de tipos básicos como de tipos referencia.

Para crear un array, podemos encontrarnos con dos situaciones:

10.1- Creación de un array con datos conocidos

Esta situación consiste en que tenemos una serie de valores conocidos y queremos hacer un array que los contenga. En este caso la creación del array es muy fácil y rápida. Por ejemplo, si queremos hacer un array que guarde varios números escribimos esto:

`int[] lista = {2,9,6,-2,0,3,7,14,28,98,5,13,-2,2,9,0,1};`


tipo de dato


variable


valores que guardamos en la lista

La variable “lista” de este ejemplo tiene tipo de dato **int[]**. La presencia de los corchetes `[]` junto a la palabra “int”, nos indica que “lista” es un array que contiene números enteros. A continuación, entre llaves se escribe la lista de números separada por comas.

Una vez que tenemos creado un array, podemos acceder a sus elementos escribiendo el nombre de la variable y encerrando entre corchetes la posición del dato al que queremos acceder (igual que hacíamos con los argumentos de la línea de comandos):


```

1. int[] lista = {2,9,6,-2,0,3,7,14,28,98,5,13,-2,2,9,0,1};
2. System.out.println(lista[3]); // muestra -2
3. System.out.println(lista[0]); // muestra 2
4. System.out.println(lista[5]); // muestra 3

```

Como vemos en el ejemplo, escribir **lista[3]** consiste en acceder al dato de la variable “lista” que está en la posición 3. Es como si “lista[3]” fuese una variable de tipo int y mostrásemos por pantalla su contenido.

De forma muy similar, si queremos cambiar el dato que está en una posición, podemos utilizar la operación de asignación así:

```

1. int[] lista = {2,9,6,-2,0,3,7,14,28,98,5,13,-2,2,9,0,1};
2. System.out.println(lista[3]); // muestra -2
3. lista[3] = 8;
4. System.out.println(lista[3]); // muestra 8

```

En este ejemplo, cuando escribimos **lista[3]=8** lo que se hace es modificar el dato de la posición 3 de la variable “lista” y cambiarlo por un 8. Igual que antes, es como si “lista[3]” fuese una variable y pudiésemos hacer asignación sobre ella para actualizar su valor.

Para mostrar por pantalla el número de datos que hay en el array, escribimos `.length`, tal y como ya se hizo en el tema 2 para contar el número de argumentos de un programa.

```

1. int[] lista = {2,9,6,-2,0,3,7,14,28,98,5,13,-2,2,9,0,1};
2. System.out.println("La lista tiene "+lista.length+" datos");

```

En todos los ejemplos que hemos visto se ha usado un array de int, pero podemos crear arrays de los demás tipos básicos y también de String, como estos:

```

1. double[] notas = {3.5, 2.9, 6, 0, 7.75};
2. boolean[] b = {true, false, true, true, false};
3. char[] vocales = {'a','e','i','o','u'};
4. String[] finde={"sábado","domingo"};

```

También podemos crear arrays que guarden otros objetos, pero debemos tener en cuenta que los objetos deben encontrarse creados previamente. Por ejemplo, el siguiente array crea una lista con tres muñecos de Mario. Observar que primero debemos crear los muñecos y luego la lista que los guarde:

```

1. Mario m1 = new Mario(100,100);
2. Mario m2 = new Mario(200,100);
3. Mario m3 = new Mario(300,100);
4. Mario[] listaMarios = {m1,m2,m3};

```

También podemos crear los muñecos directamente dentro de la lista, e incluso escribir la creación de cada muñeco en una línea diferente para que el código sea más claro:

```

1. Mario[] listaMarios = {
2.     new Mario(100,100),
3.     new Mario(200,100),
4.     new Mario(300,100)
5. };

```

Por último, indicamos que para hacer un recorrido de una lista podemos usar tanto el for tradicional que genera las posiciones de los elementos, como el for mejorado.

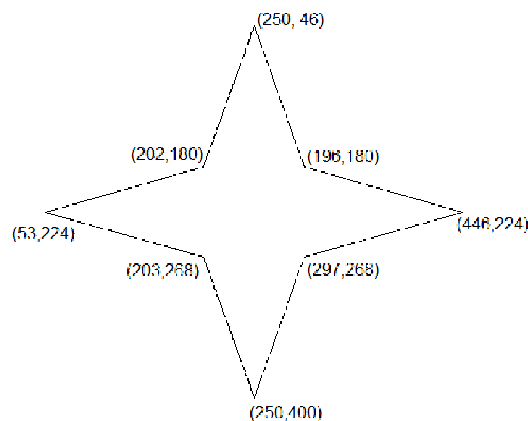
Por ejemplo, para recorrer una lista de números usando un for tradicional, creamos un for cuyo contador “i” recorra las posiciones desde 0 hasta la última (sería la anterior a lista.length) y saquemos de la lista el dato de la posición “i”:

```
1. int[] lista = {2,9,6,-2,0,3,7,14,28,98,5,13,-2,2,9,0,1};
2. for(int i=0;i<lista.length;i++){
3.     System.out.println(lista[i]);
4. }
```

Con un for mejorado, lo único que tenemos que hacer es indicar que queremos recorrer los números enteros “n” que hay en la lista:

```
1. int[] lista = {2,9,6,-2,0,3,7,14,28,98,5,13,-2,2,9,0,1};
2. for(int n : lista){
3.     System.out.println(n);
4. }
```

Ejercicio 14 : Usa la Consola DAW para realizar un programa que dibuje en pantalla el polígono de la imagen. *Sugerencia: Usa el método drawPolygon de la clase Graphics:*



Ejercicio 15 : Haz un programa que pregunte al usuario por su número de DNI. El programa mostrará la letra que corresponde a dicho DNI, teniendo en cuenta que se obtiene así:

- Calculamos el resto de dividir el número de DNI entre 23
- Buscamos en la siguiente lista la letra que corresponde a dicho resto y esa será la letra del DNI

0 → T	1 → R	2 → W	3 → A	4 → G	5 → M
6 → Y	7 → F	8 → P	9 → D	10 → X	11 → B
12 → N	13 → J	14 → Z	15 → S	16 → Q	17 → V
18 → H	19 → L	20 → C	21 → K	22 → E	23 → T

Sugerencia: Observa que cuando divides el número del DNI entre 23 te sale la posición de la tabla donde puedes encontrar la letra.

Ejercicio 16 : Realiza un programa que calcule la suma de todos estos números:

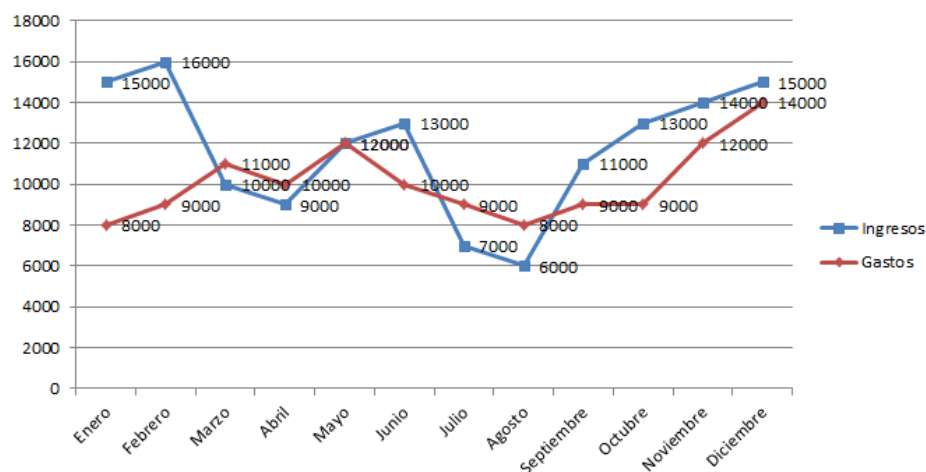
2,9,4,5,6,1,2,3,45,2,1,2,65,3,2,6,1,2,3,5,2,2,6,67,11,67,3,2,1,7,8,5,3,27,8,93,1,6

Ejercicio 17 : Realiza un programa que cree la lista de números del ejercicio anterior. A continuación, el programa preguntará al usuario: “Escribe un número” y nos mostrará en pantalla cuántas veces aparece el número en la lista y si el número está o no en la lista.

Ejercicio 18 : Realiza un programa que use 4 arrays para guardar las notas de los alumnos. A continuación, el programa mostrará el nombre de cada alumno y su nota media

Alumno/a	Programación	Base de datos	Sistemas	Nota IPE
Pepe	9	5	4	3
Ana	4	8	6	5
Juan	2	5	10	2
Roberta	10	10	10	9

Ejercicio 19 : La siguiente gráfica recoge los ingresos y gastos de una empresa en un año:



Haz un programa que muestre por pantalla:

- Si cada mes ha tenido un balance positivo o negativo.
- La media de ingresos y gastos anual
- El balance final anual y si ha sido positivo o no.

Ejercicio 20 : Usa los métodos adecuados de la clase File para realizar un programa que pregunte al usuario la ruta de una carpeta del ordenador y haga esto:

- Si la ruta introducida no es una carpeta, el programa mostrará en pantalla “Debe introducir la ruta de una carpeta” y terminará.
- Si la ruta es una carpeta, el programa mostrará por pantalla un listado de todos los archivos que hay en dicha carpeta, indicando su tamaño en bytes. No se deberán mostrar las subcarpetas.

10.2- Creación de un array con tamaño conocido

En el punto anterior hemos visto como crear un array cuando conocemos sus elementos. Pero hay muchas situaciones donde nos pasa justo al revés: no conocemos los elementos, pero si el tamaño que va a tener el array. O sea, primero hay que crear el array y luego rellenarlo.

Un ejemplo de esta situación sería: quiero rellenar un array con 50 números aleatorios. Aquí no conocemos quienes van a ser los números, pero si sabemos que van a ser 50.

En esta situación lo que se hace es crear el array de esta forma:

`int[] lista = new int[50];`

tipo de dato *variable* *tamaño del array*

Esta línea lo que hace es crear un array con 50 casillas y **las rellena todas con un valor por defecto**, que depende del tipo de dato según esta tabla:

Tipo de dato	Valor por defecto
boolean	false
int, long, short, byte	0
double, float	0.0
char	Carácter con código ASCII 0
objetos	null

Por tanto, la línea del ejemplo anterior nos crearía un array rellenado con 50 ceros. Una vez que el array está rellenado, podemos recorrer sus posiciones con un for tradicional (aquí no nos sirve el for mejorado) para actualizar dichas posiciones y asignar sus nuevos valores.

Por ejemplo, para generar un array con 50 números aleatorios haríamos esto:

```
1. // creamos un array de 50 números cero
2. int[] numeros = new int[50];
3. // recorremos el array y actualizamos cada posición con un número aleatorio
4. for(int i=0;i<numeros.length;i++){
5.     numeros[i]=(int)(10*Math.random());
6. }
```

Ejercicio 21: Consulta el pdf **arrays** y haz un programa en el que haya un array rellenado con 5000 ceros. A continuación el programa modificará las posiciones impares y guardará un 1 en ellas. Al terminar, se mostrarán todos los números del array en pantalla.

Ejercicio 22: Haz un programa que pregunte al usuario un número N y a continuación, le pida que introduzca por teclado N palabras. Al terminar, el programa mostrará todas las palabras introducidas con el siguiente formato: {palabra 1, palabra 2, palabra 3, palabra 4} y también aquellas que contienen la letra "a".

Ejercicio 23: Haz un programa que pida 6 palabras al usuario y las guarde en un array. El programa deberá mostrar por pantalla las palabras únicas (o sea, las palabras repetidas no se mostrarán).

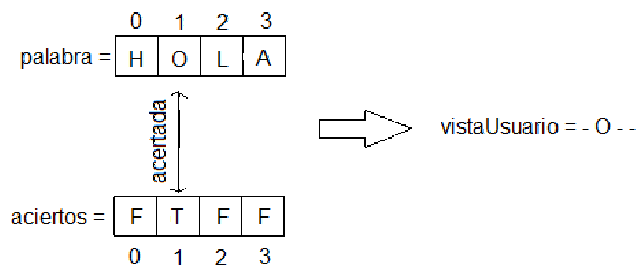
Ejercicio 24: Usa la Consola DAW para realizar un programa que pregunte al usuario un número N de vértices. A continuación, preguntará una por una las coordenadas (x,y) de cada vértice, y al finalizar, dibujará en la pantalla el polígono que tiene esos vértices. *Sugerencia: Usa el método drawPolygon de la clase Graphics*

Ejercicio 25: El juego del ahorcado

Realiza el famoso juego del ahorcado de la siguiente manera:

- Haz un ArrayList<String> llamado "palabras" en el que guardarás la lista de palabras disponibles en el juego, y una variable llamada "vidas" con el valor 4.
- Usa la clase Collections para desordenar aleatoriamente la lista, y saca la primera palabra que haya en la lista, borrándola de ella al sacarla.
- Cuenta el número de letras que hay en la variable "palabra" y haz un array de booleanos de ese tamaño llamado "aciertos", todos con el valor inicial false.

Cada booleano controla si la letra que hay en la posición correspondiente de la variable "palabra" ha sido adivinada por el usuario.



- Haz una variable String llamada "vistaUsuario" que estará formada de la siguiente forma: se recorre el array "aciertos" y por cada false, se añade un guión (-). Por cada true, se pone la letra de la variable "palabra" que se corresponde con la posición del booleano.
- Muestra por pantalla las variables "vistaUsuario" y "vidas".
- Pide una letra al usuario y comprueba si está dentro de la variable "palabra". En caso de encontrarla, pon a true todos los booleanos que se correspondan con la posición de la letra encontrada y actualiza la variable "vistaUsuario". En caso de no encontrarla, haz que el usuario pierda una vida.
- Repite los dos últimos puntos hasta que el usuario pierda o acierte la palabra. Muestra un mensaje de éxito o fracaso al final de la partida.
- Si el usuario acierta la palabra, repite el proceso sacando nuevamente la primera palabra de la lista, hasta que el usuario acierte todas las palabras o pierda sus vidas.

11.- Tablas

Como sabemos, una tabla es una estructura de datos en la que los datos se disponen formando filas y columnas. Las tablas aparecen mucho para presentar información en pantalla.



N	Documento	Emision	Vencimiento	Moneda	Total Deuda	Saldo Deuda	Es Cancelada	Pagos	Venc. Real
	NC 00100060...	2010-01-04	2010-03-05	0	-503.0	0.0	Pendiente de ...	-503.0	2010-03-05
	NC 00100061...	2010-03-16	2010-05-15	0	-3584.0	-3584.0	Pendiente de ...	0.0	2010-05-15
	NC 00100061...	2010-03-16	2010-05-15	0	-1265.0	-1265.0	Pendiente de ...	0.0	2010-05-15
	NC 00100061...	2010-03-16	2010-05-15	0	-4850.0	-4850.0	Pendiente de ...	0.0	2010-05-15
	FC 00101876...	2010-03-25	2010-05-24	0	551.0	551.0	Pendiente de ...	0.0	2010-05-24

Para trabajar con tablas usaremos nuevamente arrays, a los que añadiremos una segunda pareja de corchetes [] para representar la nueva dimensión. Por tanto, nos encontraremos las mismas dos situaciones que ya teníamos antes:

- Tenemos los objetos disponibles y creamos una tabla con ellos
- Creamos una tabla de un tamaño determinado y luego le añadimos los objetos

11.1- Creación de una tabla con datos conocidos

Supongamos que queremos introducir en la memoria este horario:

Matemáticas	Sociales	Lengua	Tecnología	Lengua
Lengua	Matemáticas	Sociales	Música	Francés
Naturales	Lengua	Matemáticas	Francés	Música
Música	Tecnología	Naturales	Matemáticas	Inglés
Inglés	Inglés	Dibujo	Sociales	Naturales
Educación física	Dibujo	Inglés	Educación física	Tutoría

El tipo de dato que utilizaremos ahora será **String[][]**. La segunda pareja de corchetes añade una nueva dimensión a los datos, de forma que ahora los primeros corchetes serán las filas y los segundos, las columnas. La tabla se introduce como una lista formada por cada fila. Podemos escribir cada fila en un renglón distinto para que sea más legible:

```
1. String[][] horario= { {"Matemáticas","Sociales","Lengua","Tecnología","Lengua"},
2.                       {"Lengua","Matemáticas","Sociales","Música","Francés"},
3.                       {"Naturales","Lengua","Matemáticas","Francés","Música"},
4.                       {"Música","Tecnología","Naturales","Matemáticas","Inglés"},
5.                       {"Inglés","Inglés","Dibujo","Sociales","Naturales"},
6.                       {"Educación física","Dibujo","Inglés","Educación física","Tutoría"}
7.                       };
```

Ahora, para acceder a un elemento deberemos incluir entre corchetes su número de fila y su número de columna (las posiciones de fila y columna empiezan por 0). Por ejemplo, así:

```
1. System.out.println("Clase del lunes a primera hora: "+horario[0][0]);
2. System.out.println("Clase del viernes a última hora: "+horario[5][4]);
3. System.out.println("Clase del miércoles a cuarta hora: "+horario[3][2]);
```

También podemos hacer modificaciones accediendo directamente a un dato de la tabla por medio de su fila y columna:

```
1. System.out.println("Clase del miércoles a cuarta hora: "+horario[3][2]); // imprime Naturales
2. horario[3][2]="Música";
3. System.out.println("Clase del miércoles a cuarta hora: "+horario[3][2]); // imprime Música
```

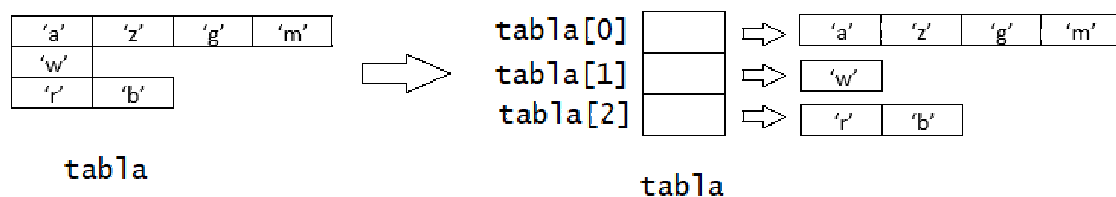
En Java podemos hacer “tablas asimétricas”, si cuando rellenamos la tabla utilizamos filas de diferente tamaño. Por ejemplo, la siguiente tabla es asimétrica:

'a'	'z'	'g'	'm'
'w'			
'r'	'b'		

Podemos crearla de la misma forma que antes:

```
1. char[][] tabla = new char[][]{
2.     {'a','z','g','m'},
3.     {'w'},
4.     {'r','b'}
5. };
```

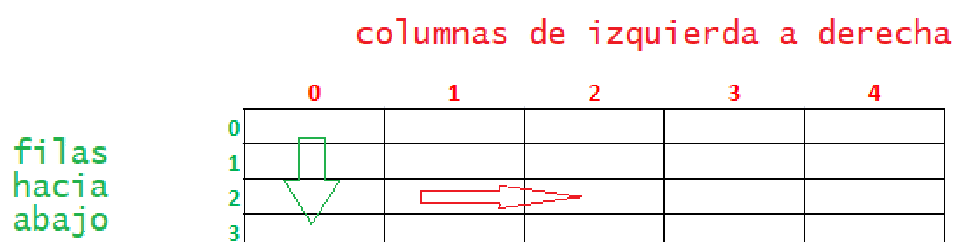
Como hemos visto en los ejemplos, en realidad una tabla no es más que una lista de listas. Podemos representar esto gráficamente así:



Por ese motivo, cuando escribimos `tabla.length` lo que obtenemos es el número total de filas de la tabla. El número de columnas de cada fila puede cambiar, pero podremos obtenerlo fácilmente escribiendo `tabla[i].length`, siendo “i” la posición de una fila.

11.2- Recorrido de una tabla

Al igual que vimos con las listas, en una tabla también podemos hacer recorridos, es decir, visitar una por una todas las casillas de la tabla. Las tablas admiten una gran variedad de recorridos: por filas, por columnas, etc. En este apartado nos centraremos en realizar un recorrido por filas, y dentro de cada fila, visitar sus columnas de izquierda a derecha.



Vamos a ver que para hacer el recorrido se necesita un bucle for anidado. En primer lugar, hacemos un for que genere las posiciones de las filas (en verde):

```
1. for(int i=0;i<tabla.length;i++){
2.
3. }
```

Este for generará los números de las filas (en verde), comenzando en 0 y terminando antes de llegar al número de filas que nos da tabla.length.

i 0
 1
 2
 3

A continuación, dentro de cada fila, debemos generar todos los números de las columnas. Estos números comienzan en 0 y terminan antes de llegar a tabla[i].length, que es el número de columnas que tiene la fila i.

Esto nos da lugar a un for anidado en el que habíamos hecho antes:

```
1. for(int i=0;i<tabla.length;i++){
2.     for(int j=0;j<tabla[i].length;j++){
3.
4.     }
5. }
```

Gráficamente, tendríamos esto:

j

i 0 0 1 2 3 4
 1 0 1 2 3 4
 2 0 1 2 3 4
 3 0 1 2 3 4

Ahora, dentro del segundo for podríamos escribir **tabla[i][j]** para recuperar el dato de la tabla que se encuentra en la fila "i" y columna "j", ya que los números que hemos generado con el for anidado nos dan justamente todas las combinaciones (fila,columna) que necesitamos para recorrer la tabla.

j		columna					fila
		0	1	2	3	4	
i	0	0	1	2	3	4	0
	1	0	1	2	3	4	1
	2	0	1	2	3	4	2
	3	0	1	2	3	4	3

Por ejemplo, para mostrar la tabla por pantalla haríamos esto:

```
1. for(int i=0;i<tabla.length;i++){
2.     for(int j=0;j<tabla[i].length;j++){
3.         System.out.println(tabla[i][j]);
4.     }
5. }
```

Ejercicio 26: Realiza un programa que cree una tabla con tu horario de clases. El programa mostrará por pantalla dicha tabla.

Ejercicio 27: Realiza un programa que cree una tabla con tu horario de clases. El programa preguntará al usuario que escriba el nombre de una asignatura, y mostrará por pantalla todos los días y horas en los que hay clase de esa asignatura.

Ejercicio 28: Realiza un programa que cree una tabla con tu horario de clases. El programa preguntará al usuario que escriba el nombre de una asignatura, y contará cuántas clases semanales se imparten de esa asignatura, y cuántas de ellas son a primera hora.

Ejercicio 29: Repite el ejercicio 18 usando un array para guardar los nombres de los alumnos, otro array para guardar los nombres de las asignaturas, y una tabla para guardar sus las notas.

Ejercicio 30: Haz un programa que cree una tabla de 4 filas y 3 columnas rellena con booleanos aleatorios. El programa mostrará la tabla en pantalla y después la recorrerá y nos dirá cuántos true y cuántos false hay en la tabla.

Ejercicio 31: Haz un programa que pregunte al usuario el número total de filas y el número total de columnas, y cree una tabla de números, en la que cada número es la suma de su número de fila y número de columna. Aquí tienes un ejemplo si elegimos 3 filas y 3 columnas:

```
Output - pruebaUnit (run) x Text Results Search Res
run:
Número de filas
3
Número de columnas
3
0 1 2
1 2 3
2 3 4
```

Ejercicio 32 : Ordenación de arrays con el método de selección

Un problema muy importante en la programación es el de ordenar de menor a mayor los números de una lista. Sigue los pasos de este ejercicio para ordenar un array usando un método llamado “método de selección”, que consiste en recorrer todas las posiciones del array, y por cada una, buscar en el resto del array el número más pequeño, intercambiándolo con aquel en el que estamos situados. Sigue estos pasos:

- Crea un array con los números 50, 26, 7, 9, 15 y 27
- Recorre todas las posiciones entre la 0 y la última y llama “i” a dicho contador
- A continuación, recorre con un nuevo bucle las posiciones desde “i+1” hasta el final del array y busca el número más pequeño que hay en esa sección del array. Deberás conservar también la posición donde se encuentra ese número.
- Intercambia el número de la posición “i” con la del número más pequeño que encontraste en el punto anterior.
- Repite el proceso con todas las “i” y tendrás el array ordenado

Gráficamente el proceso sería así:

50	26	7	9	15	27	Array original
7	26	50	9	15	27	Se coloca el 7 en primera posición. Se intercambian 7 y 50
7	9	50	26	15	27	Se coloca el 9 en segunda posición. Se intercambian 9 y 26
7	9	15	26	50	27	Se coloca el 15 en tercera posición. Se intercambian 15 y 50
7	9	15	26	50	27	El menor de los que quedan es el 26. Se deja en su posición
7	9	15	26	27	50	Se coloca el 27 en quinta posición. Se intercambian 27 y 50

11.3- Creación de una tabla con tamaño conocido

La otra situación que nos podemos encontrar a la hora de crear una tabla es que no conozcamos los elementos que vamos a guardar, pero si su tamaño. En este caso crearemos una tabla rellena con valores por defecto de forma muy parecida a los arrays:

`int[][] tabla= new int[10][20];`
↑
tipo de dato
↑
variable
↑
número de filas
↑
número de columnas

Esta línea crea una tabla con 10 filas (que se numerarían de 0 a 9) y 20 columnas (que se numerarían de 0 a 19). Toda la tabla estaría rellena con el número cero, porque cero es el valor por defecto que tiene el tipo int, tal y como aparece en la tabla de la página 13.

Una vez creada, es posible acceder a las casillas de la tabla y modificarlas de la misma forma que ya hemos visto.

Ejercicio 33: Realiza un programa que pregunte al usuario por un número N de filas y un número M de columnas. El programa creará una tabla de N filas y N columnas y las rellena con números aleatorios. A continuación, el programa mostrará la tabla por pantalla y calculará la suma de todos los números que hay en ella.

Ejercicio 34: Realiza un programa en el que haya dos arrays, uno llamado “palabras”, con 20 palabras y otro llamado “traducciones” con sus traducciones al inglés. El programa creará una tabla de 5 filas y 2 columnas, y rellena en la primera fila sus palabras en español y en la segunda las correspondientes palabras a inglés. Por último, el programa preguntará por teclado una palabra en español y nos la mostrará en inglés, usando para ello la tabla.

12.- La interfaz Set<T>

La interfaz **Set<T>** representa un conjunto de objetos de tipo T. Se diferencia de las listas en que:

- Los objetos que introducimos no se pueden repetir y solo pueden aparecer una vez.
- No hay ninguna posición para localizar a los objetos. Esto hace que la única forma de recorrer el Set<T> sea con un **for mejorado**

Como la interfaz Set<T> hereda de la interfaz Collection<T>, recibe todos los métodos que hemos visto en el apartado 4. Sin embargo, la interfaz Set<T> no define ningún método más, por lo que para trabajar con un Set<T> utilizaremos solamente los métodos que nos da Collection<T>.

Lo habitual es implementar la interfaz **Set<T>** con un **HashSet<T>** usando abstracción:

```
1. Set<String> s1 = new HashSet<>();
2. s1.add("Lunes");
3. s1.add("Martes");
4. s1.add("Miércoles");
5. for(String s:s1){
6.     System.out.println(s);
7. }
```

Si analizamos la salida del programa veremos que los días aparecen desordenados:

```
Martes
Miércoles
Lunes
```

Esto se debe a que internamente el **HashSet<T>** guarda los objetos de forma que el método **contains** (que nos dice si un objeto está o no en el Set) sea muy rápido y eficiente. Si queremos conservar el orden de los elementos introducidos, necesitaremos otras implementaciones de Set<T>, como **LinkedHashSet<T>**⁴ o **TreeSet<T>**⁵

Ejercicio 35 : Realiza un programa que cree un Set<String> llamado “mensajes” y haga esto:

- a. Lo rellene con las palabras: silla, mesa, niño, mesa, casa, silla, lápiz
- b. Muestre por pantalla el tamaño del Set
- c. Llame a su método toString y muestre por pantalla lo que produce

Ejercicio 36 : Realiza un programa que cree un **List<Long>** y un **Set<Long>** y los rellenará con 10000000 números aleatorios entre 0 y la constante Integer.MAX_LONG (cada número aleatorio se guardará tanto en la lista como en el set). A continuación, el programa preguntará un número y nos dirá si está en la lista y en el set, midiendo el tiempo empleado en consultarlo en cada uno, y se mostrará por pantalla dicho tiempo.

⁴ LinkedHashSet<T> guarda los elementos conservando el orden en el que se insertan

⁵ TreeSet<T> ordena los elementos según un criterio, como por ejemplo, el orden alfabético

13.- La interfaz Map<K,V>

Un **Map<K,V>** es una estructura que sirve para guardar dos objetos que están relacionados. Podemos imaginarnosla como si fuese una tabla con dos columnas llamadas **clave** y **valor**, de esta forma:

Clave	Valor

En la columna “clave” se guarda un objeto y en la columna “valor” se puede guardar cualquier otro objeto que esté relacionado con él. Es importante destacar **que los objetos de la columna “clave” deben ser únicos y no se pueden repetir.**

Por ejemplo, la siguiente tabla sería un ejemplo de Map<String,String> que guardaría un nombre de alumno con el nombre del curso en el que está matriculado:

Clave	Valor
Antonio López López	1º Desarrollo aplicaciones web
Jorge Ruíz Pérez	2º Desarrollo aplicaciones web
Marta López Jiménez	1º Desarrollo aplicaciones web

Observa como en este ejemplo los objetos que hay en la columna clave no se pueden repetir, mientras que la columna valor sí.

Otro ejemplo podría ser un Map<Integer,Integer> que guardase un número de teléfono con el número pin de su móvil.

Clave	Valor
611234123	1234
628765210	4953
602126511	3926

La interfaz Map<K,V> tiene muchos métodos, entre los que destacamos:

- **put:** Guarda en el Map<K,V> una asociación nueva entre un objeto clave y un valor asociado. También sirve para actualizar una asociación ya existente.
- **get:** Pasamos un objeto clave y el método nos devuelve el valor asociado a dicho objeto.
- **isEmpty:** Devuelve true si el Map<K,V> está vacío
- **size:** Devuelve el número de entradas que hay en el Map<K,V>
- **remove:** Se le pasa una clave y borra la entrada correspondiente del Map<K,V>
- **clear:** Borra todo el Map<K,V>
- **containsKey:** Pasamos un objeto clave y el método nos devuelve true si dicha clave existe en el Map<K,V>
- **containsValue:** Pasamos un objeto y el método nos devuelve true si dicho objeto está en la columna de valores del Map<K,V>.

- **keySet:** Permite obtener un Set<K> con todos los objetos clave que hay en el Map<K,V>

Para implementar **Map<K,V>** se utiliza la clase **HashMap<K,V>**

Por ejemplo, supongamos que queremos crear un Map<String,Integer> que guarda el nombre de varios puestos de trabajo y sus sueldos.

Clave (String)	Valor (Integer)
Programador	1200
Analista	1800
Jefe de proyecto	3000

Como Map<String,Integer> es una interfaz, no podemos crear objetos de ella y tenemos que acudir a alguna de las clases que hemos visto. Usamos la clase **HashMap<String,Integer>** para dar forma a nuestro Map.

Esta clase tiene un constructor sin parámetros que nos crea un Map vacío, por lo que podemos utilizarlo y a continuación añadir los objetos con el método **put**:

```
1. Map<String,Integer> m = new HashMap<>();
2. m.put("Programador",1200);
3. m.put("Analista",1800);
4. m.put("Jefe de proyecto",3000);
```

Si ahora en el programa queremos saber cuánto gana un Analista, llamaremos al método **get** pasándole la clave "Analista" y obtendremos el valor, que será el sueldo:

```
1. int sueldo = m.get("Analista");
2. System.out.println(sueldo);
```

También podemos usar el mismo método **put** para actualizar una entrada del Map. Por ejemplo, supongamos que ahora los analistas van a ganar 2000€. La actualización se haría así:

```
1. m.put("Analista",2000);
```

El resto de métodos del Map (contar elementos, borrar, etc) funcionan igual que los que ya hemos visto en las listas.

Ejercicio 37: Haz un programa que cree un Map<String,Integer> para guardar nombres de personas y sus teléfonos. El programa mostrará este menú:

1. Añadir entrada: Pregunta un nombre y un teléfono y añade esa información al Map
2. Consultar teléfono: Pregunta un nombre y, si está en el Map, muestra el teléfono correspondiente
3. Mostrar listado: Muestra un listado formado por todos los nombres y teléfonos que hay en el Map
4. Salir

Ejercicio 38 : Haz un programa en el que haya una `List<String>` para guardar títulos de películas y también haya un `Map<String,String>` en el que se va a asociar cada socio de un videoclub con la película que tiene prestada. El programa mostrará este menú:

1. Añadir película
 - Pregunta el nombre de una película y la añade a la lista de películas.
2. Añadir socio
 - Pregunta el nombre de un socio y añade al `Map` una entrada cuya clave es el nombre del socio y el valor asociado (película alquilada) es `null`.
3. Prestar película
 - Pregunta el nombre de un socio. Si el socio ya tiene una película prestada, mostrará el mensaje “El socio ya tiene una película”.
 - En caso contrario, preguntará el nombre de una película. Si la película no está en la lista de películas se mostrará el mensaje “Película no disponible”.
 - Si la película está en la lista de películas, se borrará de esa lista y se registrará en el `Map` asociada al socio.
4. Devolver película
 - Se preguntará el nombre de un socio y se mirará si tiene una película prestada.
 - En caso de no tener ninguna, se mostrará “El socio no tiene ninguna película”
 - En caso contrario, la película se añadirá a la lista de películas y en el `Map` se asociará al socio el valor `null`.
5. Salir

Ejercicio 39 : Realiza un programa en el que haya un `Map<String,String>` que guarde 20 palabras del español asociadas a sus correspondientes traducciones al inglés. El programa preguntará por teclado una palabra en español y la mostrará en inglés, usando el `Map`

Ejercicio 40 : Estos son los nombres de usuario y pin de los empleados de una empresa:

Pepe	9385
Manuel	1234
Ana	1085

Haz un programa que cree un `Map<String,Integer>` que almacene esas asociaciones. A continuación, el programa preguntará el nombre y el pin de un usuario y tras consultar el `Map` nos dirá “Acceso permitido” o “Usuario/Pin incorrecto”